答

DEFENSE CABIN · 2026
全国高校 Vibe Coding 开发挑战赛

TECHNICAL REPORT · 技术文档

答辩预演舱

多 Agent 交叉质询系统
把答辩风险提前暴露在材料里

一个面向高校答辩场景的全栈应用：上传申报书，三位 AI 评审按评审口径逐条追问，每一个问题都落回材料原文，每一条风险都能变成可执行的修改动作。

作品类型	技术栈	版本
多 Agent 协同 / 场景化 RAG / 全栈 MVP	React 18 · TypeScript · Express · SSE · BM25	v1.0 · 2026-11

ABSTRACT

摘要

答辩准备的核心矛盾不是"不知道说什么"，而是"没有人真的按评审口径逐条追问"。同学不好意思下狠手，导师时间有限，现有 AI 工具只会总结和润色。结果是材料里的弱证据——没有基线的 92.4%、没有样本量的 4 倍、把目标写成结论的 60%——在答辩现场才第一次被击穿。

「答辩预演舱」把这件事产品化：上传申报书 / 答辩稿 / 论文，系统抽取 8–12 条可质询论点，标注证据强度与 7 类缺口；方法学教授、产业评委、竞赛评审三位 Agent 进行 27 轮（高压 31 轮）交叉质询，答辩人 Agent 由材料驱动应答；最终生成五维风险雷达、风险清单、P0/P1/P2 修改清单，以及可回放到段落的质询实录。

工程上，本作品坚持三个约束：**离线可复现**（默认不依赖任何 API Key）、**结论可回放**（每个 Turn 绑定 claimId 与 BM25 证据）、**单进程交付**（评委 10 分钟内可跑通）。为此我们把"说什么"与"怎么播"拆成剧本层与编排层，用确定性规则导演保证同一材料每次生成同一条质询链，再用可选的真实 LLM 适配层提升语言质感。

27 / 31	7	5	0
标准 / 高压质询轮数	类证据缺口	维风险雷达	外部服务依赖（mock）

CONTENTS

01 问题：答辩准备的三个断层
02 产品：从材料到可执行清单
03 系统架构
04 多 Agent 协作
05 材料解析与论点抽取
06 轻量 RAG 与证据绑定
07 评分模型与报告
08 意图控制策略
09 可复现性论证
10 工程质量
11 用户体验
12 评审标准对标
13 局限与路线图
附录 API · Prompt · 复现

关键术语

术语	含义
Claim（论点）	从材料中抽出的一句可被追问的话，带类型、强度与缺口标签。
PlannedTurn	剧本层的"谁、问什么、引用哪条论点、风险级别"，不含最终台词。
Turn	播放层的最小可回放单元，含台词、证据引用、claimId。
证据芯片	追问旁的可展开标签，点开显示材料原文、章节与 BM25 分数。
高压模式	在标准 27 轮基础上追加因果归因链，共 31 轮。

01 · PROBLEM

问题：答辩准备的三个断层

我们访谈与观察了身边的答辩场景（大创、毕设、挑战杯、课程答辩），把失败原因归为三个断层。它们不是"表达能力"问题，而是"对抗性反馈缺失"。

01

没有人扮演对立面

同学旁听会给面子，导师只能提两句。材料里的"显著提升""预计降低 60%"从未被真正追问过。

02

证据缺口不可见

选手知道数字，但不知道评委在意"基线、样本量、归因、出处"。这些缺口在文字层面几乎看不出来。

03

演练不可复盘

模拟答辩说完就忘；没有记录、没有风险分级、没有"下次先改哪一条"的清单。

一个真实的材料片段

> "本项目提出一种三模态融合的实验室安全隐患识别框架，在 12 类隐患上平均识别准确率达到 92.4%。"

这句话在申报书里读起来很完整。但评审会连问三层：**跟谁比？**（基线）**多少样本？**（样本量）**跨校还成立吗？**（泛化）。如果选手没有准备，第一层就会失守。本作品要做的，是在上台前把这三层问题先问一遍。

DESIGN INSIGHT

把"文档总结"换成"论点级对抗"：不是问"这篇材料讲了什么"，而是问"这句话凭什么成立"。

02 · PRODUCT

产品：从材料到可执行清单

产品只有一条主链路，所有页面都围绕它服务。评委不需要学习任何概念，上传材料后点"开始"即可。

01	→	02	→	03	→	04	→	05
上传材料 PDF / DOCX / MD / TXT		**材料体检** 章节 / 强证据 / 缺口		**交叉质询** 3 评审 + 主持人 + 替身		**风险裁决** 雷达 / 风险 / 清单		**再练一次** 高压模式 / 版本对比

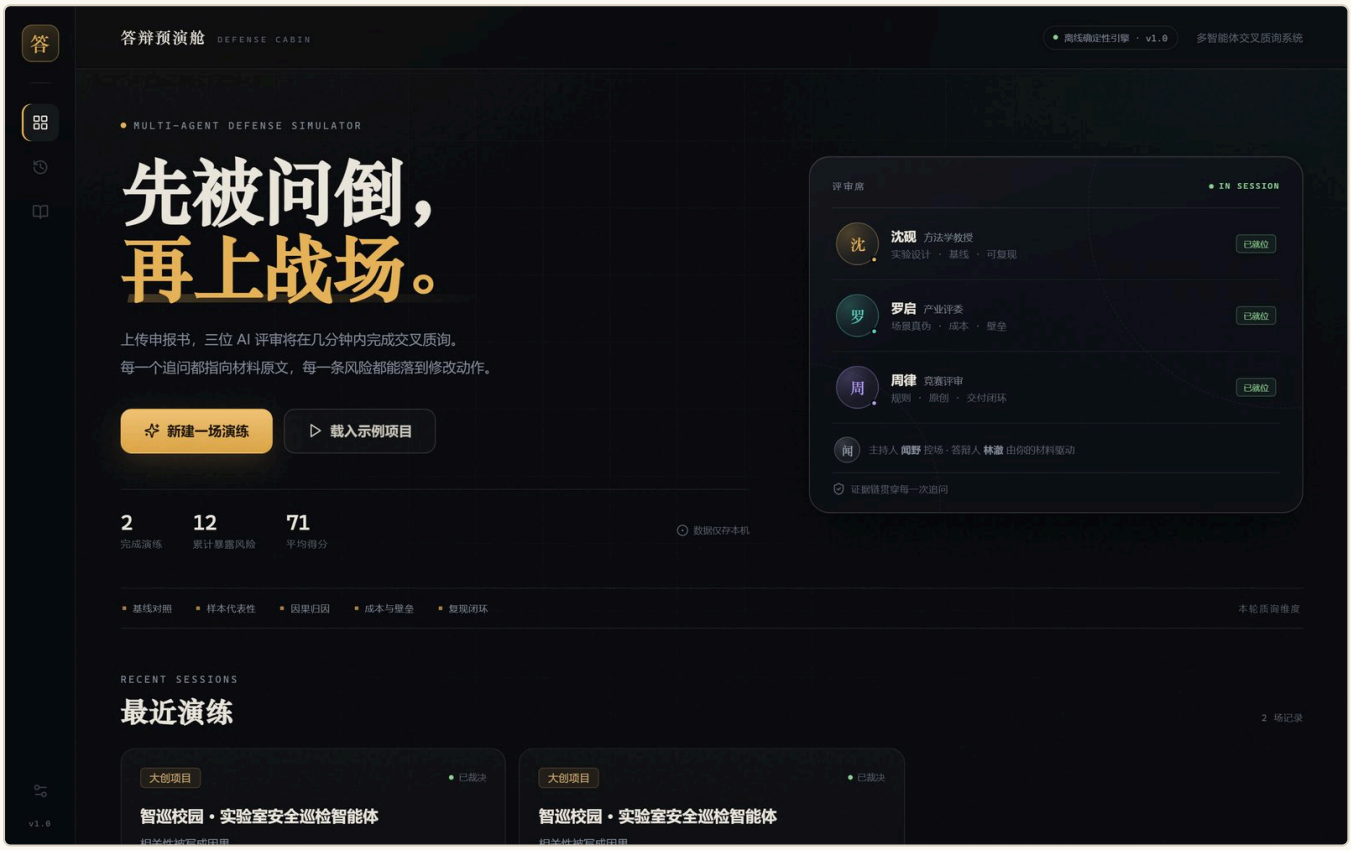


图 1 · 首页：评审席、最近演练与两入口（新建 / 示例）

首页不是"功能列表"，而是一个态度：**先被问倒，再上战场。**右侧常驻评审席让用户在上传前就知道"谁要来问我"，降低对 AI 的不确定感。

01	02	03
不替你写 只做质询与体检，不生成申报书，不替代真实评审。	**不预测奖项** 只暴露风险与修改动作，不承诺"这样改就能拿奖"。	**不追求全能** 只服务一次答辩：材料 → 质询 → 清单 → 再练。

用户旅程：五次点击完成一次闭环

阶段	用户动作	系统反馈	情绪目标
进入	打开首页，看到"评审席已就位"	3 位评审角色、最近演练、示例入口	好奇、低门槛
上传	拖入申报书，填名称 / 赛道	即时解析，显示字数、章节、论点数量	"它真的读懂了"
预检	查看材料体检与论点清单	8 个板块覆盖、强证据占比、缺口标签	紧张、有预期
质询	点击"开始交叉质询"	逐字流、轮次推进、风险实时累计、证据芯片	被追问的沉浸感
裁决	查看报告	五维雷达、6 条风险、P0 清单、逐轮回放	"知道先改什么"
再练	高压模式 / 导出 / 打印	新会话、Markdown、PDF	可行动的掌控感

关键设计：把“AI 在做什么”显性化

流式输出不是装饰。每一轮都有：谁在问（头像 / 角色 / 主问或追问）、问到哪条论点（左栏高亮）、依据是什么（证据芯片）、风险多高（红色标签）。用户不需要猜。

关键设计：报告必须能“明天就改”

每条风险都带关联论点；每条修改项都有 P0/P1/P2、工作量、预计提分。报告不是"评价"，是"施工图"。

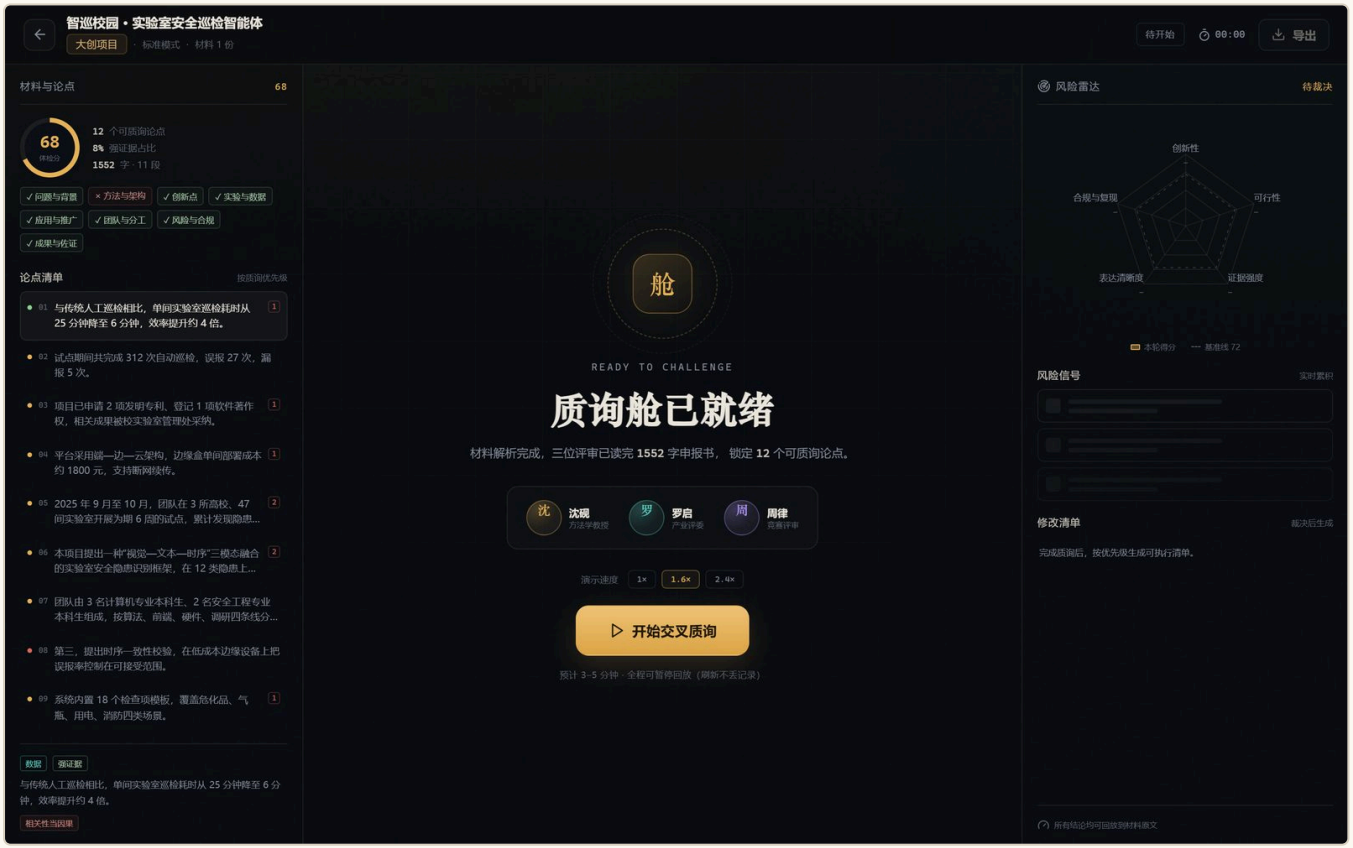


图 2 · 预检页：材料已读完，评审已就位，用户只差一次点击

系统架构：四层、一条数据流

前端不引 UI 框架，后端不引数据库。所有复杂度集中在"论点 → 剧本 → 流 → 报告"这一条链路，保证评委可读、可复现、可替换。

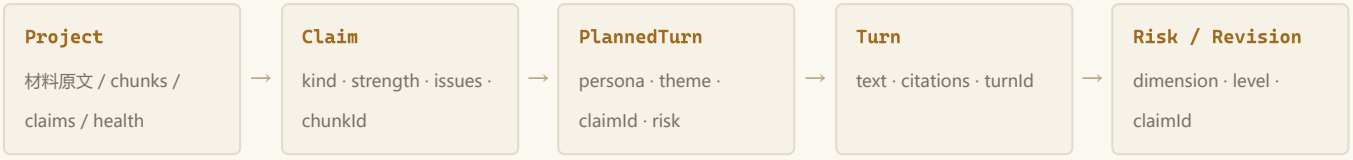


技术选型理由

选择	理由	代价 / 边界
React + Vite	上手快、构建轻、便于评委修改	无 SSR；首屏 239 kB（gzip 79 kB）
Express + SSE	流式简单、可读、无需 WebSocket 基础设施	单向流；人工双向问答需扩展
BM25（自实现）	零依赖、可解释、单文档足够	超长多文档需换向量库
JSON Store	零配置、评委可直接打开看数据	单进程；多人并发需 SQLite/Postgres
Mock 引擎	确定性、无 Key、断网可复现	不是开放推理；语言质感弱于真实 LLM

数据流：每个结论都能回溯

系统的核心不是“生成一段话”，而是维护一条可回溯的数据链。每个对象都有稳定 ID，前端可以据此高亮、跳转、导出。



为什么不用“一个大 Prompt 直接出报告”

因为不可复现、不可解释、不可局部修改。我们把任务拆成四个确定性步骤：

1. **抽取**：规则打分，产出 Claim；同一材料结果稳定。
2. **导演**：按缺口类型与角色职责生成 PlannedTurn[]；LLM 只负责润色。
3. **播放**：SSE 逐字 + 落库；刷新/断线可恢复。
4. **裁决**：按风险折算五维分数；风险与修订按标题去重。

```
interface Turn {
  id: string; round: number;
  persona: 'host' | 'method' | 'industry' | 'judge' | 'presenter';
  kind: 'opening' | 'question' | 'answer' | 'followup' | 'verdict' | 'summary' | 'system';
  text: string; citations: Citation[];
  claimId?: string; theme?: string;
  risk?: 'high' | 'medium' | 'low';
}
```

Turn 是最小可回放单元；前端所有渲染（高亮、证据、风险、报告）都从它派生。

存储片段：评委可以直接打开看

```
{
  "sessions": [{
    "id": "s-4t-Gn5pv", "status": "completed", "overall": 71,
    "turns": [{
      "id": "t3", "persona": "method", "kind": "followup",
      "claimId": "a8", "risk": "high",
      "citations": [{ "section": "3.1 三模态融合识别", "score": 4.21 }],
      "text": "也就是说，92.4% 目前既没有同参数量基线……"
    }]
  }]
}
```

04 · MULTI-AGENT

四个角色，四种下刀方式

多 Agent 不是“四个聊天框”，而是四种稳定的评审视角。每个角色有独立的关注点、语气和追问链，才能产生真正的交叉质询。

沈砚 方法学教授	**关注** 实验设计、基线、样本、消融、可复现	**口头禅** “这个数字的基线是什么？”
罗启 产业评委	**关注** 场景真伪、全生命周期成本、壁垒、付费方	**口头禅** “谁付钱？比请两个安全员便宜吗？”
周律 竞赛评审	**关注** 规则、原创边界、佐证、复现闭环	**口头禅** “专利是受理还是授权？”
闻野 主持人	**职责** 控场、记录高风险、收束裁决	**口头禅** “记录：第一条高风险。”

答辩人：材料替身

林澈不是“另一个 AI”，而是“只知道你材料里写过的东西”的数字替身。它会有理有据，也会在证据不足时露怯——这正是压力测试的意义。

为什么不用“一个模型分饰四角”

单模型容易语气同质、互相附和。我们把角色约束写进剧本与 Prompt：每个角色只问职责内的问题，追问必须绑定不同的 claim/theme。



图 3 · 质询现场：左栏论点高亮、中间逐字追问、右栏风险实时累计

质询状态机：27 轮如何组织

剧本层先生成完整轮次，再交给播放层。标准模式 27 轮，高压模式 31 轮；轮次不是固定模板，而是按材料里实际存在的缺口动态选点。

轮次	主导	主题与追问逻辑	选点规则
0	主持人	开场、规则、材料概况	固定
1	沈砚	基线 → 答辩人应答 → 追问（跨校泛化）	带 % 的 no_baseline 论点
1'	沈砚	高压追加：因果归因 → 应答 → 追问	causal 论点
2	罗启	成本 → 应答 → 壁垒追问	含成本/元/部署的论点
3	周律	原创边界 → 应答 → 复现闭环追问	创新类论点
4	沈砚	样本代表性 → 应答 → 分层追问	no_sample（排除基线/创新/承诺）
5	周律	承诺性指标 → 应答 → 佐证追问	带 % 的 promise 论点
6	三位评审	独立裁决：方法 / 产业 / 规则	固定
6'	主持人	汇总：分数、风险数、三件优先事项	由报告动态生成

高压模式加什么

在基线追问之后插入一条"因果归因"链：**"这个时间下降里，系统贡献了多少，管理流程调整贡献了多少？"** 答辩人承认没有做严格归因后，评审再追一句"那这个降幅就不能写成系统效果"。这条链是示例项目里最容易翻车的部分，也是高压模式的价值。

> **动态性：**如果材料里没有 causal 论点，高压模式不会硬加；如果没有 promise 句，承诺轮会退化为"目标与结论的边界"通用追问。所有选点都有 fallback。

一轮完整的"追问链"长什么样

```
沈砚（主问）：你写「……92.4%」。这个数字的基线是什么？
林澈（应答）：主要和上一版人工流程比，同参数量基线还在补。
沈砚（追问）：也就是说，既没有基线也没有置信区间。换成另一所学校，会掉到多少？
林澈（应答）：内部估 8–15 个百分点，但没有正式测试。
→ 系统落一条高风险；证据芯片挂回 3.1 节原文。
```

Turn 契约与 SSE 时序

流式体验的难点不是“逐字输出”，而是“刷新不丢、断线能接、消息不重复”。我们用一个简单的约定解决：先订阅，再重放；前端按 turn.id 去重。

Client	POST /sessions/:id/start	API
API	{ ok: true }	Client
Client	GET /events (EventSource)	API
API	subscribe → replay 已落库 turns	Orchestrator
Orchestrator	turn:start → turn:delta × N → persist → turn:end	Client
Orchestrator	report + done	Client

两个真实 Bug

Bug A • 丢消息

POST 返回时服务端已开始推流，EventSource 后建，前 2–3 条丢失。

修复：

连接时先 subscribe，再 replay。

Bug B • 重复

刷新先 GET 全量 turns，SSE 又 replay，start 无条件 push。

修复：

前端按 id 去重，end 覆盖同 id 占位。

```
const unsubscribe = subscribe(session.id, send);
for (const event of replayEvents(session)) send(event);
```

这个设计还有一个副作用：任何时刻刷新页面，用户看到的都是“已经说完的完整现场”，而不是半截流。对演示和评审都很重要。

材料解析：把“文章”拆成“可质询单元”

解析器不做通用摘要，而是回答一个问题：这段话里，哪一句最可能被评委追问？

01 归一化换行 / 空白 / 全角
→
02 章节 Markdown / 中文序号
→
03 切块≤620 字，保留 section
→
04 句子。！？；换行
→
05 打分数字 / 样本 / 基线 / 创新
→
06 选点12 条，类型配额

句子打分（可解释的规则）

```
score = 48
  +10 含数字            +13 含样本量（间/人/所/周/分钟/元…）
  +10 含基线词          +6  含创新词（提出/首次/自研…）
  +14 “提出…框架/方法/系统”   +4  含准确率/召回率/F1
  -14 promise   -8 vague   -7 buzzword
  -7 no_baseline  -5 no_sample  -4 no_source
```

strong ≥ 78，medium ≥ 60，weak < 60。 分数不是“文采分”，而是“被追问时的防守强度”。

排序与配额

单纯按分数排会把背景句排到第一。我们加入章节权重：背景/现状 −16，方案/数据/试点/成果 +9；再按类型配额（单一类型最多 4 条），并强制保留 1 条承诺句——因为“预计降低 60%”往往是最容易被击穿、也最容易被选手忽略的一句。

7 类缺口：把“感觉不对”变成标签

标签	触发条件（示例）	评审会怎么问
`no_baseline`	有“提升/达到/%”但没有“相比/基线/对照”	“92.4% 的基线是什么？同参数量对照有没有？”
`no_sample`	有百分比但没有样本量 / 抽样方式	“47 间实验室怎么选的？危化品占多少？”
`causal`	“由 A 缩短至 B”但没有归因 / 消融	“31 分钟里，系统贡献多少，排班调整贡献多少？”
`vague`	“显著/大幅/有效/较好”	“‘显著’具体是多少？有显著性检验吗？”
`promise`	“预计/有望/计划” + 数字	“这是目标还是结论？有历史基数吗？”
`no_source`	数字没有试点/测试/统计/附件出处	“这个数字的原始记录在哪？能抽查吗？”
`buzzword`	≥3 个概念词堆叠	“去掉这些词，你的原创贡献是什么？”

强度不是“好坏”，而是“防守难度”

强证据（有数字 + 样本 + 基线）不等于创新强；弱证据也不等于项目差。它只表示：这句话在质询中需要更多支撑。报告的语言也遵循这个原则——不评价项目价值，只标注证据状态。

为什么用规则而不是模型

规则可解释、可单测、可复现；模型抽取会随温度漂移，评委无法复现同一结果。我们把“抽取”交给规则，把“表达”留给可选 LLM。



图 4 · 左栏：12 条论点、强度点、缺口计数；当前质询论点高亮

轻量 RAG：BM25 + 证据芯片

本作品不需要向量库。单文档、1–2 万字、强调可解释，BM25 是更合适的选择：零依赖、可复现、每个分数都能解释。

中文分词

字符 unigram + bigram：既保留单字召回，又捕捉"基线""样本""复现"这类双字术语；英文与数字按词保留；去掉"的/了/在/是/我们/通过"等停用词。

BM25

$$\text{score}(q,d) = \Sigma_{t \in q}\ \text{IDF}(t) \cdot \frac{f(t,d)\cdot(k_1+1)}{f(t,d)+k_1\cdot(1-b+b\cdot|d|/\text{avgdl})}$$

$k_1 = 1.4 \cdot b = 0.75 \cdot$ 每条追问取 top 2 · 无命中回退到论点所在 chunk

每条追问会带 1–2 条引用；前端渲染为"证据 · 章节"芯片，点开后显示原文、BM25 分数、chunkId。这样评委不需要相信模型，只需要看原文。



图 5 · 证据芯片：问题旁边直接挂材料章节，点开可核对原文

07 · SCORING

评分模型：锋利，但不失控

评分不是“AI 主观打分”，而是风险折算。每个扣分项都能回跳到具体风险，每条风险都能回跳到论点。设计目标是“让选手知道先改什么”，而不是“给一个漂亮分数”。

维度	权重	基准分	主要扣分来源
创新性	22%	79	buzzword、原创边界不清
可行性	22%	76	成本、团队、场景
证据强度	26%	76	no_baseline / no_sample / causal
表达清晰度	14%	83	vague、promise
合规与复现	16%	77	no_source、权重/数据未闭环

RISK → SCORE

$$\text{deducted} = \Sigma\,\{\text{high}:7, \text{medium}:4, \text{low}:2\}$$

$$\text{score} = \text{clamp}(\text{base} - \min(16, \text{deducted}), 48, 93)$$

$$\text{overall} = \Sigma\, \text{score}_i \times \text{weight}_i$$

单维最多扣 16 分；风险按标题去重；修订项按标题再去重

两次校准

第一版把示例项目打到 66 分、证据强度 42，还出现三条同名“缺少基线”。这不是严格，是模型不可信。第二版把高风险扣分从 9 降到 7、加 16 分封顶、按标题去重，示例项目稳定在 71 分，证据强度 60——仍然指出问题，但不会把有真实试点的项目“判死刑”。

> “评分的价值不是排序，而是让选手相信：我按这个清单改，下一次会更好。”

07 · SCORING

报告：从“评价”到“施工图”

报告页是整个产品的收束。它必须在 30 秒内回答三个问题：我哪里最危险？为什么？先改什么？



图 6 · 报告首屏：总分、五维雷达、三条 P0 行动

A

风险清单

每条风险带等级、维度、关联论点；可按“仅高风险”筛选。

B

修改清单

P0/P1/P2、工作量、预计提分、关联论点；可勾选完成。

C

质询实录

按轮次折叠，保留每一句追问与证据，可回放到具体段落。

报告同时支持 Markdown 导出与浏览器打印 PDF。打印样式会隐藏交互控件、切换为白底，方便直接附在提交材料里。

报告章节与数据来源

章节	回答的问题	数据来源
评审结论	整体怎么样？先做什么？	五维分数 + P0 修订项
五维雷达	短板在哪？	风险按维度折算，单维封顶 16
风险清单	为什么危险？	被质询论点的 issue + 追问轮次
修改清单	具体怎么改？	issue → 动作/工作量/预计提分
质询实录	当时到底问了什么？	Turn 全量落库，含证据引用

意图控制：五层约束，让 Agent 不跑偏

“会写 Prompt”不是把要求堆成一大段，而是把角色、任务、证据、格式、边界分开约束。每一层都对应一个可验证的失败模式。

层	内容	约束 / 失败模式
角色层	姓名、头衔、关注点、语气、口头禅	只问职责内的问题；避免四个角色语气同质
任务层	当前 theme、目标 claim、轮次、追问深度	一次只问一件事；避免“连环十问”
证据层	论点原文 + BM25 片段 + 章节	必须引用；禁止编造数字/文献
格式层	80–120 字、中文、无姓名前缀	避免舞台说明、避免 Markdown 标题
安全层	不人身攻击、不预测奖项、不替代真实评审	拒绝越界；报告页有免责声明

上下文预算

真实 LLM 模式每次只带：**项目名称 + 当前论点 + 最近 4 轮 + 本轮任务**。不塞完整材料，避免长上下文漂移；事实由 BM25 片段提供，结构由剧本提供。

失败回退

```
AI_MODE=openai && OPENAI_API_KEY
  → POST /chat/completions
  → 非 200 / 超时 / 空内容
  → return null
  → orchestrator 使用 script.ts 的原始台词
```

因此真实模型是增强，不是单点依赖。评委没有 Key、断网、接口超时，都不影响完整演示。

Prompt 实例：让追问“像人”，但不失控

角色 System Prompt（真实 LLM 模式）

```
你在扮演高校答辩模拟评审「沈砚·方法学教授」。
说话锋利、具体、只说一件事，不堆术语，不重复材料原句。
只输出一段中文台词，80–120 字，不要加姓名前缀，不要解释。

项目：智巡校园 · 实验室安全巡检智能体
论点：本项目提出一种“视觉–文本–时序”三模态融合的……
最近对话：
  沈砚：这个指标的基线是什么？
  林澈：主要和上一版人工流程对比……
请按原计划的意思，用你的口吻说：
  “也就是说，92.4% 目前既没有同参数量基线，也没有置信区间……”
```

为什么把“原计划”也给模型

让模型润色，而不是让模型决定问什么。剧本层已经决定了“问基线、问跨校泛化”，LLM 只负责把这句话说得更像沈砚。这样即使模型漂移，质询链的结构仍然稳定。

反例：不要这样写

```
你是一个严格的评审，请根据材料提出 10 个深刻的问题，
覆盖创新性、可行性、商业价值、技术路线、团队、风险，
每个问题要有深度，最好能难住选手。
```

这种 Prompt 会得到“正确的废话”：问题很多、很全、但无法回放到具体论点，也无法评分。结构不是靠形容词堆出来的。

09 · REPRODUCIBILITY

可复现性：把“过程”也当作交付物

赛题把“过程可复现”放在 25% 的第一位。我们的理解是：评委不需要相信我们的叙述，只需要能重跑、能对照、能抽查。

证据	内容	位置
确定性引擎	mock 模式无随机数；同材料 → 同剧本 → 同评分	`script.ts` / `scoring.ts`
细粒度 Git	20+ 次提交，按模块/修复拆分，非“initial commit”	`git log --oneline`
Prompt 链	5 个核心迭代：意图、产出、翻车、修复、验证	`docs/prompts/`
Bug 修复	年份被吃掉、SSE 丢/重、评分 42、构建冲突	`docs/prompts/02~05`
10 分钟路径	install → build → start → demo	`README.md` 第 3 节
可视化回放	应用内“研发现场”时间线	`/making-of`

10 分钟复现（实测）

```
pnpm install      # 2–5 min，取决于网络
pnpm build        # 约 10 s
pnpm start        # 单进程 :8787
# 打开 http://127.0.0.1:8787 → 载入示例 → 开始交叉质询
```

边界：首次安装依赖 npm 网络；完全断网环境无法保证首次安装。安装完成后，运行与演示可完全离线。

四个真实 Bug：从翻车到修复

这些不是"为了文档编的例子"，而是开发过程中真实出现、定位、修复并加测试的问题。每个都能在 Git 历史中找到对应提交。

Bug	现象	根因 → 修复
年份被吃掉	"2025 年 9 月"渲染成"年 9 月"	清洗正则把任意开头数字当列表序号 → 只匹配 1–2 位 + . 或 、
核心创新句落榜	12 条论点里没有 92.4% 那句	数据类配额满 + 分类被 % 抢走 → "提出…框架"直判创新，提高权重
SSE 丢/重	前几条消息丢失；刷新后重复	先推流后连接 / 无条件 push → 先 subscribe 再 replay，按 id 去重
评分 42 + 三连重复	证据强度 42；三条同名风险/P0	扣分无上限 + 无去重 → 单维封顶 16，按标题去重

修复示例：清洗正则

```
- .replace(/^[-•·*\d.\s、]+/, '')
+ .replace(/^[-•·*]\s*/, '')
+ .replace(/^\d{1,2}[.、]\s*/, '')
```

修复示例：评分封顶

```
- const PENALTY = { high: 9, medium: 5.5, low: 3 };
+ const PENALTY = { high: 7, medium: 4, low: 2 };
- value -= PENALTY[risk.level];
+ deducted += PENALTY[risk.level];
+ value -= Math.min(16, deducted);
```

工程质量：能维护，才敢叫“作品”

AI 生成的代码最容易死于“能跑但没人敢改”。我们用三层约束控制：类型契约、单一数据访问层、可测试的纯函数。

层	职责	关键文件
Shared	前后端共享类型、Persona 常量、StreamEvent	`packages/shared`
Domain	切块、分词、BM25、论点抽取、体检	`domain/text.ts` · `analyze.ts`
Engine	剧本、编排、SSE、评分、LLM 适配	`engine/*.ts`
Interface	REST、上传、导出、静态托管	`routes.ts`
Store	JSON 原子写、启动恢复	`store.ts`

安全与边界

- 上传限制 20MB / 6 个；仅解析 pdf/docx/md/txt；不执行宏。
- 文件只在内存解析，不按原始文件名落盘；避免路径穿越。
- React 默认转义；全项目不使用 `dangerouslySetInnerHTML`。
- 材料只写本机；mock 模式无出站请求；报告可本地删除。
- Prompt 明确“不预测奖项、不替代真实评审”；页面有免责声明。

API 契约

```
GET  /api/health                  → { ok, mode }
POST /api/projects                → Project (multipart)
POST /api/sessions                → Session
POST /api/sessions/:id/start      → { ok }
GET  /api/sessions/:id/events     → SSE
GET  /api/sessions/:id/report.md  → text/markdown
POST /api/demo                    → { projectId, sessionId }
```

测试与性能：用数据说话

7/7	0	27	71
单元测试通过	E2E console error	标准轮次	示例项目综合分

测试覆盖

测试	断言
年份保留	“2025 年”原样出现在论点文本中
创新句入榜	含 92.4% 的句子必须被抽取，且带 no_baseline
promise 兜底	至少一条承诺句进入质询清单
体检边界	score ∈ (30, 94]；8 个覆盖项
BM25	查询“试点/实验室/响应时间”命中 47 间段落
风险去重	titles 无重复；修订项无重复
分数边界	总分 50–90；每维 48–93

性能预算

指标	实测 / 目标
材料解析（1.5k 字）	< 50 ms
BM25 建索引	< 10 ms
27 轮 SSE	约 20–40 s（speed=1，可调 1–4×）
前端首屏 JS	239 kB / gzip 79 kB
CSS	49 kB / gzip 10 kB
冷启动	< 1 s（无数据库/外部服务）

这些数字不惊艳，但足够支撑“评委在笔记本上 10 分钟跑完”的目标。我们优先选择可复现，而不是极限性能。

体验：让“被追问”变得可看、可读、可承受

答辩模拟天然让人紧张。视觉设计不能加重压迫感，也不能做成轻佻的聊天玩具。我们选择“深夜评审舱”：暗色、克制、琥珀色重点，像一间真的评审室。

Ink	Ivory	Amber	Cyan	Violet	Risk
#090B0F	#ECE7DD	#E8B45A	#5FD3C4	#B79CFF	#E06C5E

三个体验决策

1. 先给“谁在问”。头像、姓名、头衔、主问/追问标签先于正文出现，用户不会迷失在文字里。

2. 风险实时长出来。右栏在质询过程中从骨架变成具体风险，制造“问题正在累积”的紧迫感。

3. 证据随手可查。每个问题旁都有证据芯片，点开就是原文；不需要信任 AI。

动效原则

只用在“状态变化”上：消息进入、流式光标、风险累计、雷达生长。不做无意义的漂浮和炫技；截图与录屏都要能看清信息。



图 7 · 移动端：390px 下仍保持标题层级与双入口

评审标准对标

维度	权重	本作品的证据	可现场验证
过程可复现 与演进链路	25%	20+ 次细粒度提交；5 个 Prompt 链；4 个真实 Bug 修复；mock 确定性；10 分钟复现	`git log` / `docs/prompts` / `/making-of`
解题有效性 与实用价值	25%	真实答辩痛点；上传→质询→报告→再练闭环；P0 修改清单可直接执行	载入示例跑完整场
方法创新性 与意图控制	20%	论点级多 Agent 交叉质询；剧本/播放分离；五层 Prompt 约束；证据绑定	打开证据芯片 / 看 `script.ts`
工程质量 与架构设计	15%	四层架构、共享类型、原子写、SSE 重放、7 个单测、无数据库	`pnpm test` / 代码结构
用户体验 与产品精细度	15%	暗色评审舱、逐字流、风险实时、响应式、打印样式、空状态	质询中刷新 / 移动端

如果评委只给 3 分钟

1

看首页

“先被问倒，再上战场”——一句话说清产品态度。

2

跑示例

点“载入示例 → 开始”，看 30 秒质询与证据芯片。

3

看报告

五维雷达 + 3 条 P0，证明不是“聊天玩具”。

诚实边界：示例项目“智巡校园”为虚构演示材料；产品支持上传真实材料，但评分仍是演练参考，不构成奖项预测。

13 · LIMITS

局限与下一步

一个 10 分钟能跑通的作品，一定做了取舍。我们把边界写清楚，而不是假装它是通用平台。

当前局限	影响	下一步
Mock 引擎是规则导演	语言质感弱于真实 LLM，复杂学科问题覆盖有限	接入真实模型 + 领域问题库；保留 mock 作为复现基线
仅支持可复制文本	扫描版 PDF 无法解析	接入 OCR（PaddleOCR / 云 OCR）
JSON 单进程存储	多人并发不适合	SQLite / Postgres + 用户体系
BM25 单文档	长论文、多附件检索弱	pgvector / Qdrant + 混合检索
评分未分学科	文科/工科/医学口径不同	赛道模板：权重、问题库、证据规则可配置
无真人对练	答辩人由 AI 替身应答	人工模式：用户输入回答，评审动态追问

路线图

v1.1
人工答辩模式 + 语音输入

v1.2
赛道模板与评分权重配置

v1.3
两次演练 diff：分数 / 风险 / 修订完成度

v2.0
多文档知识库 + 真实评审圆桌

CLOSING

答辩不是把做过的事讲一遍，
而是证明你的结论经得起别人换一个角度再问一次。

答辩预演舱 · Defense Cabin · 2026

APPENDIX A

附录：API 与数据模型

StreamEvent

```
type StreamEvent =
  | { type: 'turn:start'; turn: Turn }
  | { type: 'turn:delta'; id: string; delta: string }
  | { type: 'turn:end'; turn: Turn }
  | { type: 'status'; status: SessionStatus; progress: number }
  | { type: 'report'; scores; risks; revisions; summary; overall }
  | { type: 'done' }
  | { type: 'error'; message: string };
```

Session

```
interface Session {
  id: string; projectId: string; projectName: string;
  status: 'ready' | 'running' | 'completed';
  mode: 'mock' | 'llm'; intensity: '标准' | '高压';
  turns: Turn[]; scores: DimensionScore[];
  risks: RiskItem[]; revisions: Revision[];
  summary: string; overall: number; progress: number;
  createdAt: string; completedAt?: string; speed: number;
}
```

文件布局

```
apps/server/src/domain/    text.ts · analyze.ts · sample.ts
apps/server/src/engine/    script.ts · orchestrator.ts · scoring.ts · llm.ts
apps/server/src/           routes.ts · store.ts · report.ts · config.ts
apps/web/src/pages/        Dashboard · SessionRoom · ReportPage · MakingOf
packages/shared/src/       index.ts (类型 + Persona 常量)
```

APPENDIX B

附录：Prompt 索引与复现命令

#	文件	关键内容
01	`docs/prompts/01-scope.md`	赛题收敛、产品边界、AI 不能代做的部分
02	`docs/prompts/02-material-parser.md`	论点抽取、年份 Bug、创新句落榜
03	`docs/prompts/03-orchestrator.md`	多 Agent 编排、SSE 丢/重、刷新恢复
04	`docs/prompts/04-scoring.md`	评分校准、风险去重、P0 清单
05	`docs/prompts/05-build.md`	pnpm junction、esbuild 双版本、可复现构建

复现命令

```
pnpm install
pnpm build
pnpm start
# http://127.0.0.1:8787

pnpm test     # 7 个单测
pnpm e2e      # Playwright 全链路
```

关键环境变量

```
PORT=8787
AI_MODE=mock          # mock | openai
OPENAI_BASE_URL=https://api.openai.com/v1
OPENAI_API_KEY=       # 可留空
OPENAI_MODEL=gpt-4o-mini
DEMO_SPEED=1
```



先被问倒，再上战场。

答辩预演舱 · Defense Cabin